eman ta zabal zazu

Universidad del País Vasco
Euskal Herriko Unibertsitatea

*Sistemas de Gestión de Seguridad de Sistemas de Información*

# Sistema Web


Diego Pomares

Ivan Salazar

Lou Marine Gómez

David Miguez


*29/10/2025*

Ingeniería Informática de Gestión y Sistemas de Información

# ÍNDICE

1. Introducción....3
2. Primera Auditoría....4
2.1. ZAP....4
3. Vulnerabilidades....5
3.0. Rotura de Control de Acceso....5
3.1. Inyección SQL....5
3.2. Fallos criptográficos....5
3.2.1. Almacenamiento de contraseñas....5
3.3. Diseño inseguro....6
Directory Browsing....6
3.4. Configuración de seguridad insuficiente....6
3.4.1. Content Security Policy header....6
3.4.2. Clickjacking....6
3.4.3. X-Content type....7
3.5. Componentes vulnerables y obsoletos....7
3.5.1. Info Leaking....7
3.6. Fallos de identificación y autenticación....7
3.6.1. CSRF Token....7
3.6.2. Bruteforce....7
3.6.3. HTTP Only flag....7
3.7. Fallos en la integridad de datos y software....8
3.8. Fallos en la monitorización de la seguridad....8
4. Bibliografía....8

# 1. Introducción

La finalidad de esta fase del proyecto es analizar y evaluar el nivel de seguridad del sistema web desarrollado inicialmente. Con el propósito de cumplir con los estándares de seguridad actuales, esta etapa se centra en la aplicación de herramientas y técnicas de auditoría exploradas en clase.

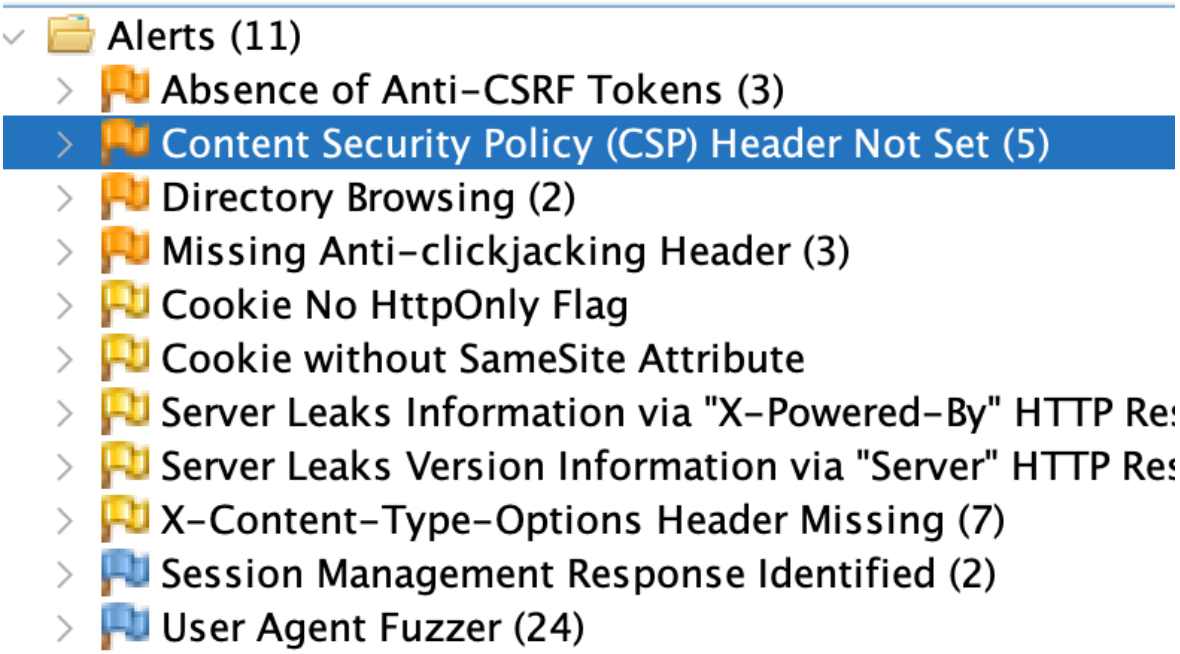
En cuanto a las herramientas utilizadas, destacamos el uso de OWASP ZAP (Zed Attack Proxy), complementado con una auditoría manual exhaustiva del código fuente. Este análisis nos ha permitido poner a prueba la infraestructura de autenticación, el control de acceso y la gestión de datos sensibles, señalando fallos de lógica y de configuración que podrían ser aprovechados por un atacante, poniendo en riesgo los datos y la privacidad de los usuarios del sistema.

Este proceso nos ayuda a aplicar las habilidades y tecnologías dadas en clase y, sobre todo, servirá como punto de partida para la siguiente etapa del proyecto, donde se diseñarán y aplicarán las técnicas necesarias para mejorar la seguridad y robustez del sistema.

# 2. Primera Auditoría

## 2.1. ZAP

Como ya hemos mencionado antes, para llevar a cabo un análisis automático de las vulnerabilidades más comunes, hemos utilizado **OSWAP ZAP (Zed Attack Proxy),** una herramienta de seguridad de aplicaciones web de código abierto.
Al ejecutarla en nuestro sistema web, obtuvimos el siguiente listado de errores y fallos de configuración:




<u>Dónde se habla de cada vulnerabilidad:</u>
CSP-header & X-Content-Type/Samesite & Clickjacking **-> Configuración de seguridad insuficiente**
Directory Browsing -> **Diseño inseguro**
Server Leaks via * -> **Componentes vulnerables y obsoletos**
CSRF & HTTP-Only -> **Fallos de identificación y autenticación**

# 3. Vulnerabilidades

## 3.0. Rotura de Control de Acceso

Mediante el uso de $_SESSION[“user”] se evita la manipulación de la URL para por ejemplo modificar datos de usuarios diferentes al usuario que no acaba de iniciar sesión. Si en lugar se generasen IDs mediante $_GET[“user”] se comprometería la información de otros usuarios.

```
require 'connect.php';
$id = $_SESSION["user"];
```

## 3.1. Inyección SQL

Mediante el uso de bind_param en la conexión a la base de datos se evitan inyecciones de código de SQL, el cual de otra manera se podría inyectar introduciendo entre comillas en los campos de registro o inicio de sesión y otros.

```
$res = $conn->prepare("SELECT * FROM usuarios WHERE nombre_usuario=?");
$res->bind_param("s", $usuario);
$res->execute();
```

## 3.2. Fallos criptográficos

### 3.2.1. Almacenamiento de contraseñas

Las contraseñas de nuestro sistema se almacenan utilizando password_hash(). Esta función hashea la contraseña y además con sal para guardarla en la base de datos.

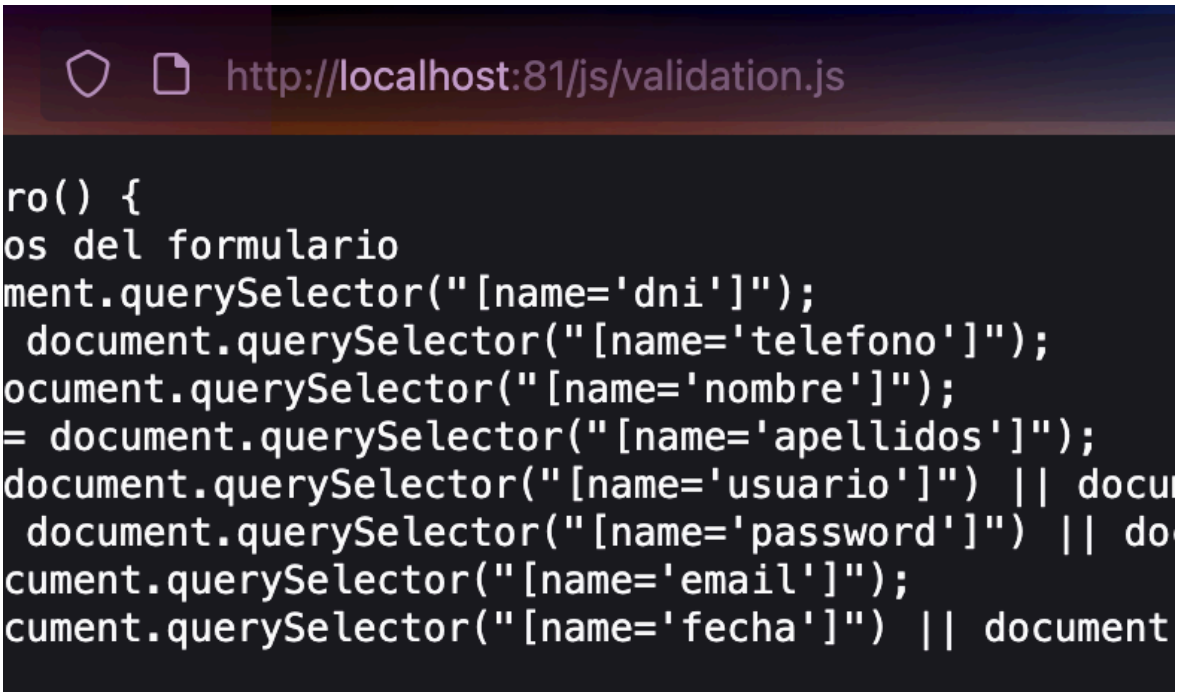
```
$pass = password_hash($raw_pass, PASSWORD_BCRYPT);
```

Para mayor seguridad se podría implementar métodos de encriptación de datos más complejos con SSL por ejemplo.

## 3.3. Diseño inseguro

### Directory Browsing

Es la manipulación de la url que cualquiera en la página puede usar para ver los diferentes directorios e incluso ver código fuente por ejemplo en la carpeta /css o /js.




## 3.4. Configuración de seguridad insuficiente

### 3.4.1. Content Security Policy header

Aunque ZAP no ha alertado de vulnerabilidades de cross-site scripting, no hemos implementado headers CSP los cuales son cabeceras que indican al usuario qué tipo de cosas puede cargar a la página y así limitarlo para que no se inyecte código malicioso. Esto reduce considerablemente las posibilidades de que cualquier atacante inyecte código que pueda interactuar directamente con el backend.

### 3.4.2. Clickjacking

El atacante usa varias capas transparentes u opacas para engañar al usuario y hacer que haga click en algo diferente a lo que ve, lo que lo puede llevar a realizar acciones no deseadas en la aplicación vulnerable.
En nuestro sistema no tenemos cabeceras “X-Frame-Options” o “Content-Security-Policy: frame-ancestors”, esto hace que sea vulnerable a este tipo de ataque. El atacante podría aprovechar acciones como modificar datos de usuario para robar sus datos.

### 3.4.3. X-Content type

La cabecera X-Content-Type-Options no se encuentra configurada en las respuestas del servidor. Esta cabecera evita que el navegador intente adivinar el tipo de contenido de los archivos, lo cual puede derivar en la ejecución de código malicioso si un archivo es interpretado incorrectamente. Incluir X-Content-Type-Options: nosniff asegura que el navegador respete el tipo MIME declarado y reduce el riesgo de ataques de inyección o ejecución no deseada de recursos.

## 3.5. Componentes vulnerables y obsoletos

### 3.5.1. Info Leaking

Mediante cabeceras como X-Powered-By o Server, la conexión revela información sobre los componentes y versiones de estos. ZAP ha detectado cabeceras que revelan el uso de php 7.2.2 y de apache 2.4.25. Esto podría ser información que utilice un atacante astuto para explotar vulnerabilidades de estas herramientas y versiones.

```
Date: Thu, 30 Oct 2025 17:16:00 GMT
Server: Apache/2.4.25 (Debian)
X-Powered-By: PHP/7.2.2
```

## 3.6. Fallos de identificación y autenticación

### 3.6.1. CSRF Token

Esta medida es esencial para verificar que las peticiones provienen del usuario legítimo y no han sido generadas desde un sitio externo. La falta de validación CSRF permite que un atacante manipule acciones autenticadas en nombre del usuario sin su consentimiento. Implementar un token único y no predecible en cada solicitud POST o acción sensible mitiga completamente este tipo de ataque.

### 3.6.2. Bruteforce

Esto no lo indica ZAP, pero no se ha implementado ninguna medida para evitar que se automaticen ataques de bruteforce. Esto se puede lograr fácilmente generando tiempos de espera largos tras varios intentos fallidos, lo cual destruye la eficiencia de cualquier algoritmo de bruteforcing y le haría tardar mucho tiempo.

### 3.6.3. HTTP Only flag

ZAP ha detectado que las cookies de sesión no están configuradas con las banderas HttpOnly ni SameSite. La falta de HttpOnly permite que el contenido JavaScript del cliente pueda acceder a las cookies, aumentando el riesgo de robo de sesión mediante ataques XSS. Por otro lado, la ausencia del atributo SameSite posibilita que navegadores acepten

solicitudes entre sitios sin restricciones, lo que puede facilitar ataques de tipo CSRF. Añadir ambas banderas en la configuración del servidor refuerza la seguridad de las cookies al limitar su exposición y su envío a contextos seguros.

## 3.7. Fallos en la integridad de datos y software

Aunque nuestra web no está actualizando sus dependencias constantemente, es importante asegurarse de que los datos y actualizaciones de software utilizado (apache, php, etc) sea de fiar y de esa manera proteger la integridad del software. Además, proteger la integridad de cierta manera también incluye protegerla de errores introducidos por los propios desarrolladores (nosotros) por lo que se puede implementar una Pipeline CI/CD y reglas de ramas en github.

## 3.8. Fallos en la monitorización de la seguridad

No hay ningún mecanismo para anticiparse a amenazas de seguridad y errores. Esto fácilmente se puede lograr creando copias de seguridad con cierta frecuencia y escribiendo logs cuando saltan errores en lugares importantes para que futuros desarrolladores puedan tener contexto para identificar problemas más eficientemente.

# 4. Bibliografía


- Apuntes de Mikel
  https://github.com/mikel-egana-aranguren/EHU-SGSSI-01/tree/main/Seguridad_web
- Wikipedia CSRF
  https://es.wikipedia.org/wiki/Cross-site_request_forgery
- Mozilla Developer Network
  https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP
  https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers
- web.dev
  https://web.dev/articles/security-headers?hl=es
- OWASP
  https://owasp.org/www-project-top-ten/

- ChatGPT
  https://chatgpt.com/